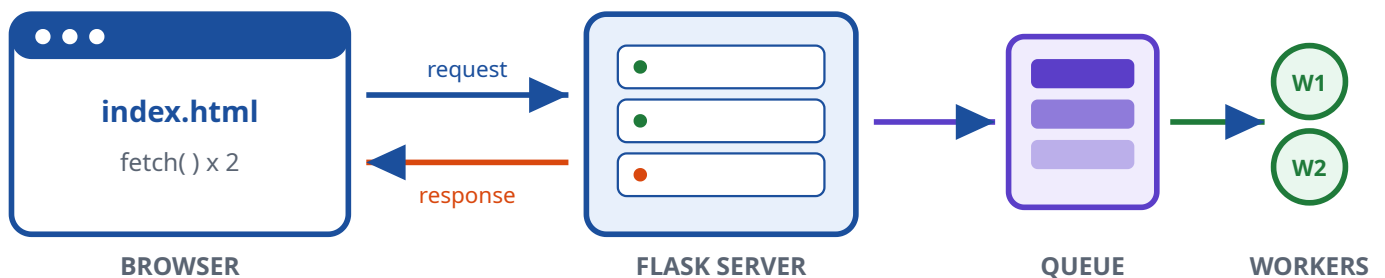


MODERN APPLICATION DEVELOPMENT II

Week 9 – Graded Assignment 9 Complete Illustrated Solutions

Flask timing • Redis • Celery • Concurrency • Message Queues & Brokers

Everything in this assignment, in one picture



Written for a complete beginner. If you have never seen a line of code before, start at Part 1 and read straight through. Every idea is explained with a picture and a real-life story before any code appears.

READ THIS FIRST

How to use this book

This document does **two** jobs at the same time:

1. **It teaches you the subject from absolute zero.** Parts 1 to 7 build your knowledge brick by brick, with pictures and everyday stories.
2. **It solves all 10 questions** of the assignment, step by step, showing exactly how a marker expects you to reason.

THE PROMISE OF THIS BOOK

Imagine you are 10 years old and you have never touched a computer. Every hard word (server, thread, broker, TTL) will first be explained using things you already know: a restaurant, a school queue, a lunch box, a notebook, a post box. Only after the story do we look at the code.

Map of the book

Part	What you learn	Questions solved
Part 1	The internet in slow motion: client, server, request, response, Flask, routes	background
Part 2	Reading the clock in code: <code>time.sleep</code> and <code>datetime.now().second</code>	background
Part 3	JavaScript <code>fetch</code> and promises: how the browser asks two questions at once	background
Part 4	One waiter or many waiters: <code>threaded=True</code> vs <code>threaded=False</code>	Q1 and Q2
Part 5	Redis: the super-fast notepad that lives in memory	Q3 and Q4
Part 6	Celery: sending slow homework to a helper	Q5
Part 7	Concurrency vs Parallelism: juggling vs having two hands	Q6 and Q7
Part 8	Message queues and message brokers: the post box and the post office	Q8, Q9, Q10
Part 9	One-page cheat sheet, full answer key, and practice questions	revision

The four kinds of boxes you will see

STORY BOX

A real-life picture of the idea. Read this one first, always.

DEFINITION BOX

The proper technical wording, the one you should write in an exam.

TRAP BOX

The exact mistake that makes students lose the mark. Learn these by heart.

ANSWER BOX

The final answer of a question, with the option letter.

A NOTE ON HONESTY

In the screenshots you shared, questions 3 to 10 clearly display the green "*Answer is Correct*" banner, so those answers are confirmed. Questions 1 and 2 only show a *selected* option (both marked **B**), with no correctness banner. Question 1 really is **B**. Question 2 is **not B** - and the whole point of that question is to catch exactly that mistake. Part 4 shows you, second by second, why the answer changes. Please read it slowly.

The internet in slow motion

Before we can answer a single question, we must be sure about one thing: what actually happens when a web page "loads"? Let us slow it down 1000 times.

STORY BOX • THE RESTAURANT

You walk into a restaurant and sit down. You are the **customer**. You call the waiter and say "one dosa please". That sentence is your **order**. The waiter walks to the kitchen, the cook makes the dosa, and the waiter brings the plate back to your table. That plate is the **reply**.

The whole internet works exactly like this. Nothing more.

- You (the customer) = the **browser**, also called the **client**. Chrome, Firefox, Safari.
- The kitchen = the **server**. A computer somewhere that knows how to make things.
- Your order = a **request**.
- The plate that comes back = a **response**.

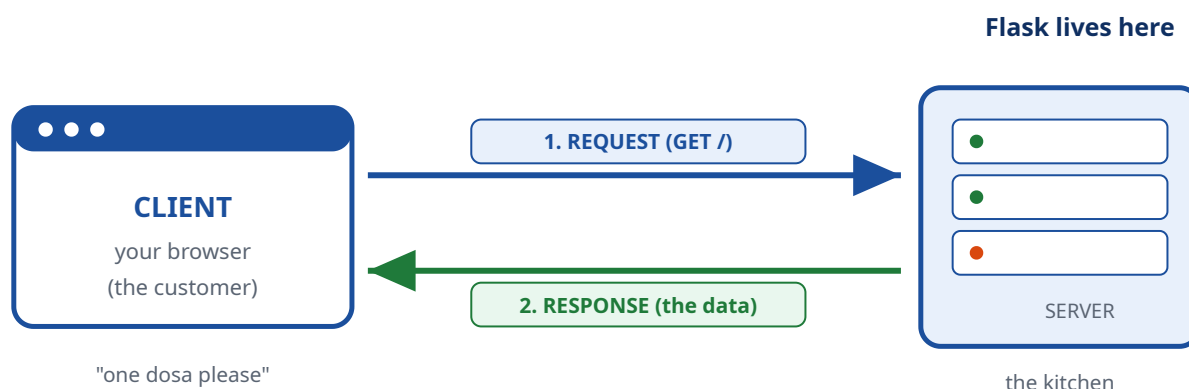


Figure 1.1 – Request and response. The client always asks first; the server only answers.

DEFINITION BOX

A **client** sends a **request** to a **server**. The server does some work and sends back a **response**. The server never speaks first. This ask-then-answer pattern is called the **request-response cycle**.

1.1 What is a web address really saying?

In the assignment you see the address `http://127.0.0.1:5000/profile`. It looks scary. It is just four pieces of information, like a postal address.

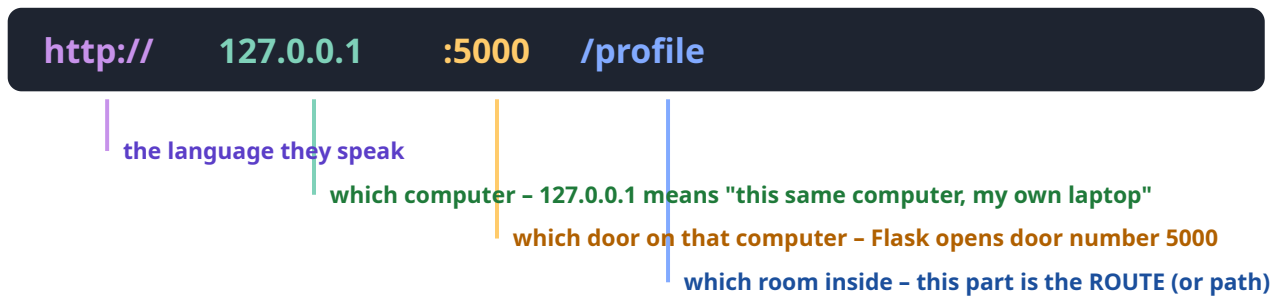


Figure 1.2 – Anatomy of a URL. `127.0.0.1` is nicknamed *localhost*: your own machine.

STORY BOX • THE BUILDING

127.0.0.1 is the building (your own house). **5000** is the door number of one particular office in that building. **/profile** is the desk inside that office. Change the desk and a different person answers you.

1.2 What is Flask?

Flask is a small Python toolkit that turns your Python file into a kitchen that can answer orders. You write ordinary Python functions, and you stick a label on each function saying "call me when somebody asks for *this* address".

```
# the smallest possible Flask app
from flask import Flask    # borrow the toolkit

app = Flask(__name__)      # create the kitchen

@app.route('/')             # the LABEL: "when someone asks for / ..."
def home():                 # ... run this function
    return "Hello!"         # and send this back as the response

if __name__ == "__main__":
    app.run()               # open the kitchen, start listening
```

DEFINITION BOX • THREE WORDS TO KEEP

- **Route:** the address part after the port, for example `/` or `/profile`.
- **View function:** the Python function that runs for that route (`home` , `profile`).
- **Decorator:** the line starting with `@` . It is the sticky label that connects a route to a view function.

1.3 The two rooms in our assignment

Our `app.py` has exactly two rooms. One is slow. The other is *very* slow. Everything in Q1 and Q2 depends on this difference, so let us draw it.

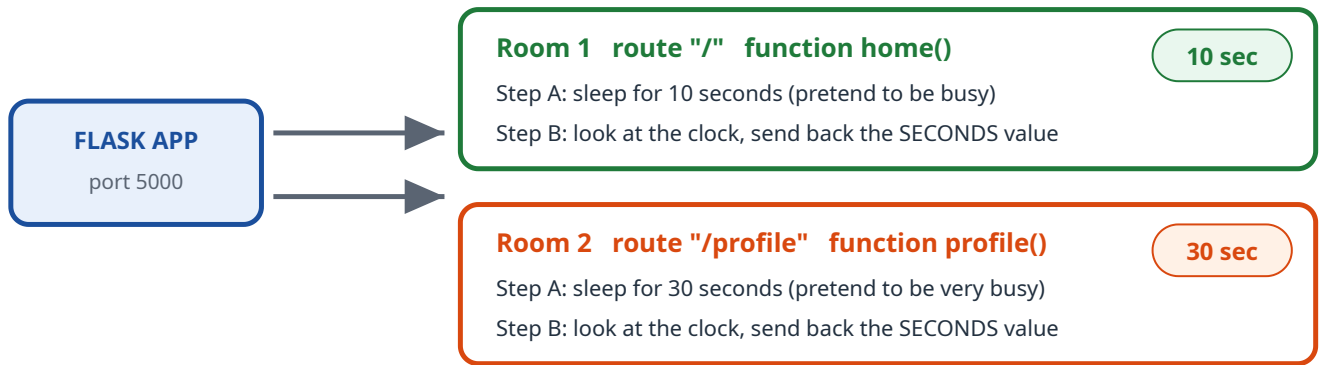


Figure 1.3 – The two routes. Notice the clock is read **after** the sleep, never before.

Telling the time inside code

Two lines of Python do all the interesting work in Q1 and Q2. Let us take them apart.

2.1 `time.sleep(10)` – the freeze button

STORY BOX • THE STATUE GAME

Your teacher shouts "STATUE!" and you must freeze completely for 10 seconds. You are not thinking, not moving, not talking. When she shouts "MOVE!" you carry on from exactly where you stopped.

`time.sleep(10)` is the word STATUE for a program. The program stops on that line, does nothing at all for 10 seconds, then continues to the next line.

Why would anyone deliberately waste 10 seconds? Because in a real app that line would be something genuinely slow: reading a huge file, training a model, generating a PDF report, calling another company's server. `sleep` is a stand-in that lets us practise with slow work without writing the slow work.

2.2 `datetime.now().second` – the "seconds hand only"

Read it from left to right, like a sentence:

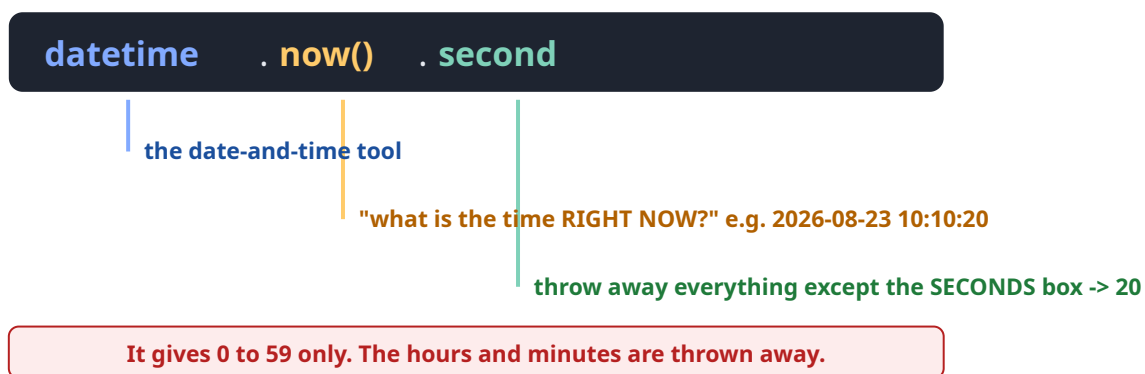


Figure 2.1 – Reading `datetime.now().second` as an English sentence.

Worked examples

Clock time	.hour	.minute	.second	What the code returns
10:10:10	10	10	10	10
10:10:20	10	10	20	20
10:10:40	10	10	40	40
10:10:50	10	10	50	50
10:11:05	10	11	5	5 (not 65! the seconds hand wrapped around)

TRAP BOX

Students often compute "10 + 30 = 40 seconds of waiting, so the answer is 40" without checking where the clock started. Always do it in two moves: **(1)** find the wall-clock time when that line runs, **(2)** read only the seconds part.

2.3 The golden order of the three lines

```
@app.route('/')
def home():
    time.sleep(10)           # 1. FIRST freeze for 10 s
    return jsonify(datetime.now().second), 200 # 2. THEN read the clock
```

THE SINGLE MOST IMPORTANT SENTENCE IN THIS ASSIGNMENT

The clock is read **after** the sleep finishes. So the number you get is **(time the function started) + (sleep duration)**, expressed as seconds only.

And the extra bits on that return line

Piece of code	What it means, in plain words
<code>jsonify(...)</code>	Wrap the value in JSON, the common text format that browsers and servers both understand. <code>jsonify(20)</code> travels across the wire as the text <code>20</code> .
<code>200</code>	The HTTP status code for "everything went fine". (404 = not found, 500 = server crashed.)
<code>{'Access-Control-Allow-Origin': '*'}</code>	A permission slip called CORS . By default a browser refuses to let a page read data from a different address. The <code>*</code> means "anyone may read me". Without this line the fetch would be blocked and you would see a CORS error instead of a number. It does not change any timing.
<code>debug=True</code>	Developer comfort: show full error pages and auto-restart when you edit the file. Also does not change the timing logic.

How the browser asks two questions at once

Now we move from the kitchen (Python) to the customer (JavaScript inside `index.html`). Here is the whole script again, with every line numbered so we can talk about it.

```
// 1 function test() {
// 2   const res1 = fetch('http://127.0.0.1:5000')    // order sent NOW
// 3   const res2 = fetch('http://127.0.0.1:5000/profile') // order sent NOW too
// 4
// 5   res1
// 6     .then((res) => { return res.json() }) // when plate 1 arrives, unwrap it
// 7     .then((data) => { console.log(data) }) // then print it
// 8
// 9   res2
//10     .then((res) => { return res.json() }) // when plate 2 arrives, unwrap it
//11     .then((data) => { console.log(data) }) // then print it
//12 }
//13 test() // run it immediately when the page opens
```

3.1 What is `fetch` ?

STORY BOX • TWO ORDERS, ONE TRIP TO THE WAITER

You and your friend sit at the same table. You call the waiter once and say: "one dosa AND one biryani". The waiter writes both slips and walks off.

You do not wait for the dosa before ordering the biryani. Both orders are **already in the kitchen** within the same second. This is exactly what lines 2 and 3 do: two orders are placed back to back, in the same instant.

DEFINITION BOX

`fetch(url)` sends the HTTP request **immediately**, and instantly hands you back a **Promise**: a receipt token that says "your food is coming, hold on to this". JavaScript does not stop and wait. It runs the very next line straight away.

3.2 What is a Promise, really?

STORY BOX • THE FOOD-COURT TOKEN

At a food court you pay and receive a **plastic token with a number**. The token is not your food. It is a promise of food. You can walk around, order at a second counter, chat with friends. When your token buzzes, you collect the food.

`res1` and `res2` are two tokens. `.then(...)` means "when this token buzzes, do this". Writing `.then` does not make the food arrive faster; it only says what to do *on arrival*.

Both requests leave the browser at the SAME moment (10:10:10)

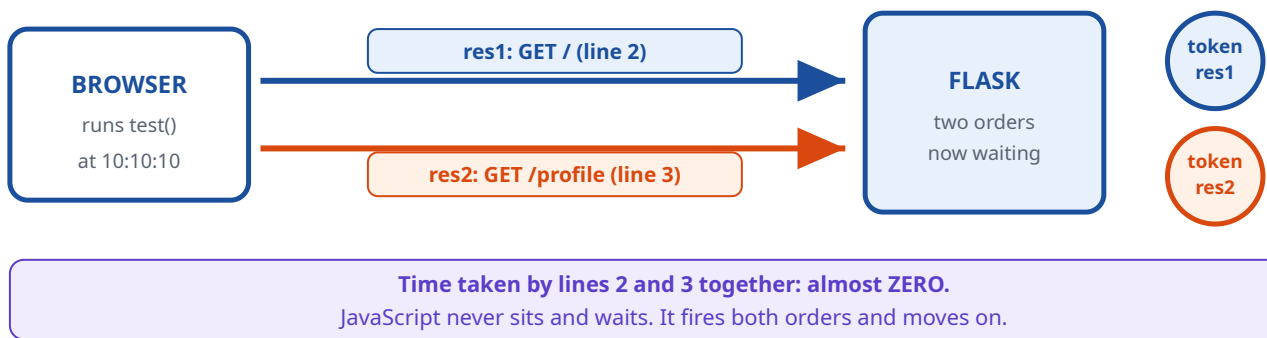


Figure 3.1 – Two fetches fired in the same instant, two promise tokens returned instantly.

3.3 Why there are two `.then` calls per request

The reply does not arrive as a finished number. It arrives as a sealed parcel. Unwrapping takes two moves:

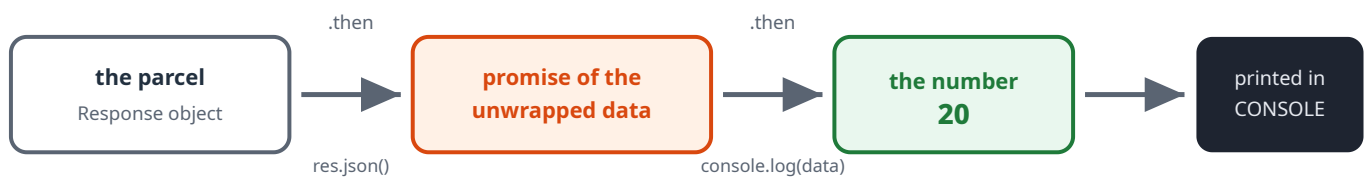


Figure 3.2 – First `.then` unwraps the JSON, second `.then` prints it. This unwrapping is instant; it adds no waiting time.

3.4 The rule that decides the ORDER of the console output

GOLDEN RULE

The console prints in the order the **replies arrive**, not in the order the code is written. Whoever finishes first, prints first.

This one rule is what the whole of Q1 and Q2 tests. In both questions `res1` finishes before `res2`, so `res1`'s number is printed first. But **which numbers** appear depends on how many waiters the server has. That is Part 4.

SMALL PRINT, SO NOBODY CAN TRICK YOU

The word "approximately" in the question is there because network travel and `res.json()` take a few milliseconds. A real run might show 21 instead of 20 if the machine is slow. In the exam you always ignore those milliseconds and use the clean arithmetic.

One waiter, or many waiters?

Q1 and Q2 print the same `app.py` and the same `index.html`. Only **one** thing changes between them:

Question 1 has:

```
app.run(debug=True)
```

Question 2 has:

```
app.run(threaded=False, debug=True)
```

That one extra word, `threaded=False`, changes the answer. To see why, we need to understand what a **thread** is. Story first, as always.

4.1 What is a thread?

STORY BOX • THE WAITER IN THE RESTAURANT

A **thread** is one worker who can carry out one job at a time, start to finish.

Restaurant A (many waiters). A new customer walks in, and the manager assigns a *fresh waiter* to that table. Ten customers, ten waiters. Your slow order does not block anybody else's order.

Restaurant B (exactly one waiter). There is one waiter for the whole restaurant, and he refuses to touch a second table until the first table is fully served. Everybody else stands in a line at the door, waiting their turn.

Both restaurants serve identical food. The only difference is **how long you wait**.

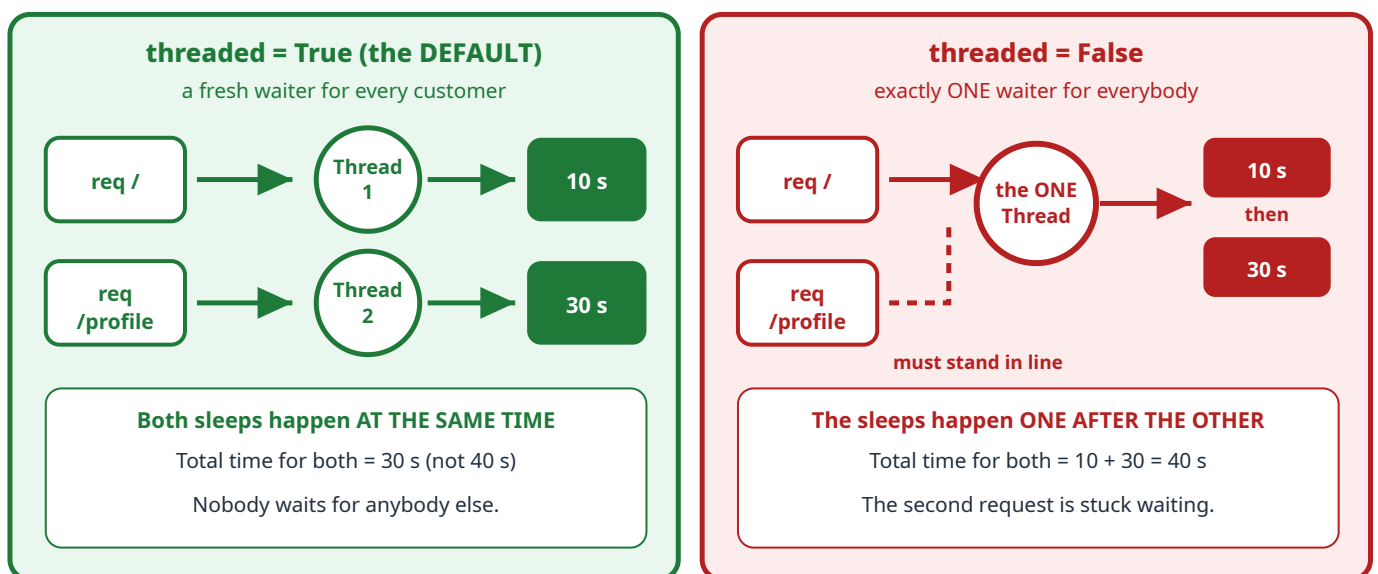


Figure 4.1 – The only difference between Q1 and Q2, drawn as two restaurants.

THE FACT YOU MUST MEMORISE

When you call `app.run()` in Flask, Flask quietly sets **`threaded=True` for you**. So plain `app.run(debug=True)` already handles requests **at the same time**. You get the one-waiter behaviour **only** if you switch it off yourself by writing `threaded=False`.

This has been the behaviour since **Flask 1.0**. In much older versions you had to write `threaded=True` yourself. Underneath, Flask hands your settings to the Werkzeug development server, whose own default was `threaded=False` – which is why the fact is worth memorising rather than guessing.

TRAP BOX

Many students believe the opposite: that Flask is single-threaded until you ask for threads. That belief gives the wrong answer to Q1. Remember the direction: **Flask's default is already threaded**. **`threaded=False` is a downgrade you choose**.

Question 1

1 mark

With `app.run(debug=True)`, a user visits `index.html` at **10:10:10 AM**. What will be logged in the console (approximately)?

A. 20, 50

B. 20, 40

C. 40, 40

D. 50, 50

Step-by-step solution

1 Check the server mode.

The code is `app.run(debug=True)`. There is no `threaded` word, so Flask uses its default, `threaded=True`. This is **Restaurant A: many waiters**. Requests are served at the same time.

2 Set the starting clock.

The page opens at **10:10:10**. `test()` runs instantly, so both `fetch` calls leave at 10:10:10. Call this moment **$t = 0$** .

3 Follow request 1, the one to `/`.

Its own waiter starts at $t = 0$, sleeps 10 s, so it reaches the return line at $t = 10$, which is the wall-clock time **10:10:20**.

`datetime.now().second` reads **20**. It sends 20 back.

4 Follow request 2, the one to `/profile` .

It has its *own separate* waiter, who also starts at $t = 0$ (it did not have to wait for request 1). It sleeps 30 s, reaching the return line at $t = 30$, which is **10:10:40**.

`datetime.now().second` reads **40**. It sends 40 back.

5 Decide the printing order.

Request 1's reply lands at $t = 10$; request 2's reply lands at $t = 30$. Earlier arrival prints first, so the console shows **20** and then **40**.

Q1 TIMELINE - threaded=True: the two bars OVERLAP

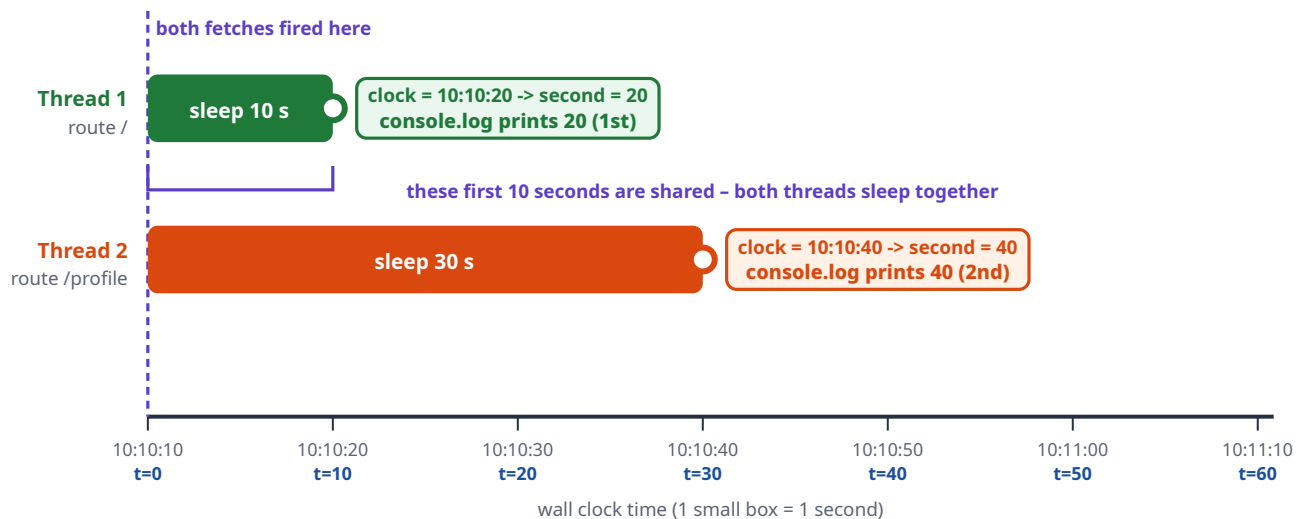


Figure 4.2 – Q1. Both bars start at $t = 0$ because each request gets its own thread.

ANSWER TO QUESTION 1

Console output: **20, 40** => **Option B**

Because both requests are handled simultaneously: 10 s after 10:10:10 gives second 20, and 30 s after 10:10:10 gives second 40.

Why the other three options are wrong

Option	The wrong belief behind it
A. 20, 50	This assumes <code>/profile</code> had to queue behind <code>/</code> ($10 + 30 = 40$ s of waiting, landing at 10:10:50). That is true only when threading is switched OFF. This is the answer to Q2, not Q1.
C. 40, 40	This assumes both routes finish at the same moment. They cannot: one sleeps 10 s and the other 30 s.
D. 50, 50	Mixes both mistakes at once: queueing <i>and</i> equal finish times.

Question 2

1 mark

Same files, but now the last line is `app.run(threaded=False, debug=True)`. A user visits `index.html` at **10:10:10 AM**. What will be logged in the console (approximately)?

A. 20, 50

B. 20, 40

C. 40, 40

D. 50, 50

IMPORTANT – PLEASE READ BEFORE THE SOLUTION

In your screenshot, option **B (20, 40)** is the one that was selected for this question, and there is no green "Answer is Correct" banner on it. Q1 and Q2 cannot both be **B**: the examiner changed the code on purpose so the answer would change. The correct answer here is **A (20, 50)**. The next page proves it second by second.

Step-by-step solution

1 Check the server mode – this is the whole question.

We now have `threaded=False`. This is **Restaurant B: exactly one waiter**. The server can hold only **one** request at a time; a second request must wait at the door until the first is completely finished.

2 Decide who gets served first.

Both requests were fired at 10:10:10, but not in a random order. Line 2 of the script fires `/` and line 3 fires `/profile`, so `/` reaches the server a fraction of a second earlier. The single waiter therefore picks up `/` first, and `/profile` stands in the queue.

3 Serve request 1 (`/`).

Starts at $t = 0$, sleeps 10 s, reaches the return line at $t = 10$, wall clock **10:10:20**. Sends back **20**. Identical to Q1 so far – the first request never notices the difference.

4 Now serve request 2 (`/profile`) – here is the change.

The waiter is free only at $t = 10$. **Only now** does `profile()` begin. It then sleeps 30 s, so it reaches the return line at $t = 10 + 30 = t = 40$.

$t = 40$ after 10:10:10 is the wall clock time **10:10:50**.

`datetime.now().second` reads **50**. It sends back **50**.

5 Printing order.

Reply 1 lands at $t = 10$, reply 2 lands at $t = 40$. So the console prints **20** first, then **50**.

Q2 TIMELINE – threaded=False: the bars CANNOT overlap

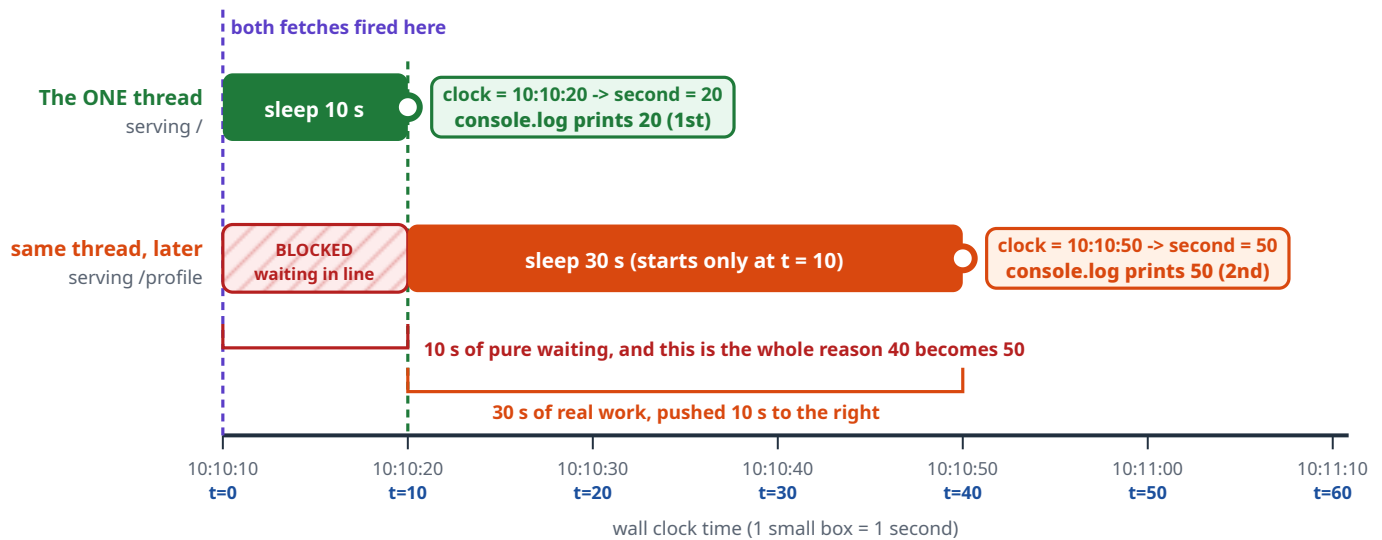


Figure 4.3 – Q2. The orange bar is shifted 10 seconds later because the single thread was busy.

ANSWER TO QUESTION 2

Console output: **20, 50** => **Option A**

With one thread the work is serialised: 10 s for `/`, then 30 s for `/profile`, so the second reply is read at $10 + 30 = 40$ s after 10:10:10, i.e. 10:10:50.

A neat cross-check the examiner built in

Suppose you doubted step 2 and thought `/profile` might be served first. Then `/profile` would finish at `t = 30` (second = 40) and `/` would finish at `t = 40` (second = 50), so the console would print **40, 50**. Look at the option list: **40, 50 is not offered**. The only way to land on a listed option is the order we used: `/` first, then `/profile`. The options themselves confirm the reasoning.

4.2 Q1 and Q2 side by side

	Q1 – <code>app.run(debug=True)</code>	Q2 – <code>app.run(threaded=False, ...)</code>
Threads available	many (Flask default is <code>threaded=True</code>)	exactly one
<code>/</code> starts at	<code>t = 0</code> (10:10:10)	<code>t = 0</code> (10:10:10)
<code>/</code> finishes at	<code>t = 10</code> (10:10:20) => logs 20	<code>t = 10</code> (10:10:20) => logs 20
<code>/profile</code> starts at	<code>t = 0</code> (no waiting)	<code>t = 10</code> (waited 10 s)
<code>/profile</code> finishes at	<code>t = 30</code> (10:10:40) => logs 40	<code>t = 40</code> (10:10:50) => logs 50
Total time to finish both	30 s	40 s
Console	20, 40 (Option B)	20, 50 (Option A)

SAY IT TO YOURSELF IN ONE BREATH

"Many waiters, both start together, so 20 and 40. One waiter, the second one has to queue, so 20 and 50."

THE REUSABLE FORMULA FOR EVERY QUESTION OF THIS TYPE

Answer second = (start second + everything you had to wait for + your own sleep) mod 60

- Threaded server: "everything you had to wait for" = 0.
- Single-threaded server: it is the total sleep time of all requests ahead of you in the queue.
- The `mod 60` just means: if you go past 59, wrap around to 0 again.

Redis: the lightning-fast notepad

5.1 First, why do we need Redis at all?

STORY BOX • THE ALMIRAH AND THE STICKY NOTE

Imagine you are doing homework. Your big textbook is inside a locked steel almirah across the room. Every time you need a fact you must stand up, walk over, unlock it, flip pages, walk back. Very reliable, very slow.

So you copy the four facts you need again and again onto a **sticky note next to your hand**. Now looking them up is instant.

- The steel almirah = a normal **database** on the **disk** (like SQLite, MySQL, Postgres).
- The sticky note = **Redis**, living in **RAM**, right next to the processor.

A sticky note has a catch, though: if somebody opens the window, it blows away. RAM is the same – switch the power off and it is gone. Speed is bought with fragility.

5.2 RAM vs Disk: the size of the speed difference

People say "RAM is faster". You must feel *how much* faster, because that single fact is why Redis exists.

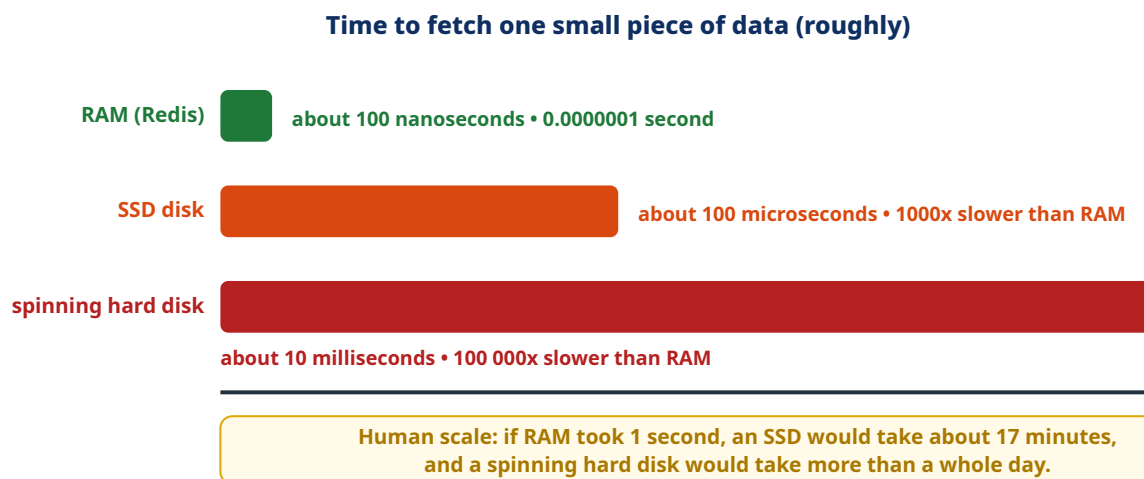


Figure 5.1 – Why "keep it in memory" is such a big deal. Bars are not to scale; the real gap is even bigger.

5.3 What does "key value pair" mean?

STORY BOX • THE SCHOOL BAG TAG

In a cloakroom every hook has a **name tag**, and one bag hangs on it. Say the tag out loud and you instantly get the right bag. You never search through all the bags.

The tag is the **key**. The bag is the **value**. Redis stores nothing but tag-and-bag pairs. This is why Redis is called a **key-value store** and not a table-based (relational) database. There are no rows, no columns, and no `JOIN` statements.

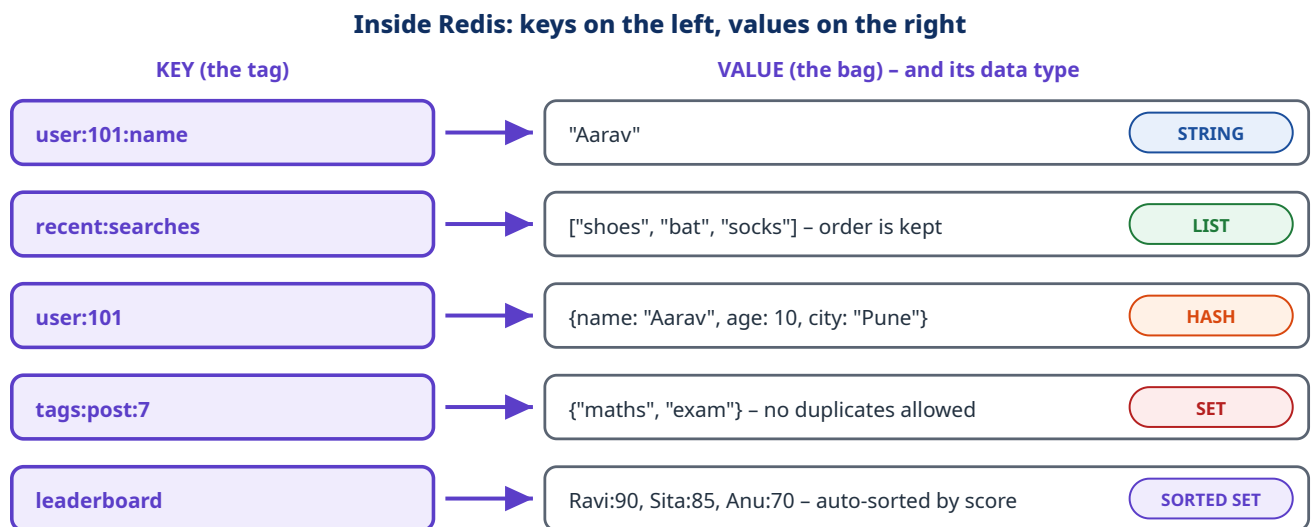


Figure 5.2 – Redis keys are always text, but values come in several shapes. This is what "supports multiple data types" means.

5.4 If RAM forgets everything, can Redis ever recover?

Yes – and this matters for one of the options. Redis can copy its memory onto the disk now and then, purely as a **backup**, so that after a restart it can refill RAM.

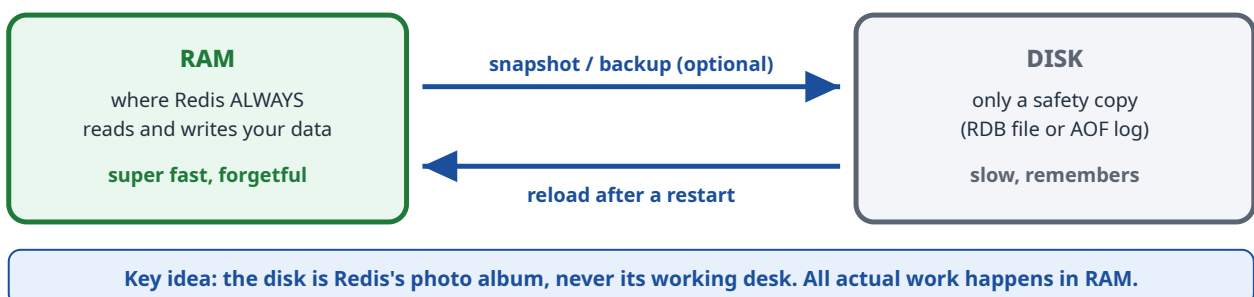


Figure 5.3 – Persistence in Redis is a backup mechanism, not the place data lives.

Question 3

1 mark • multiple correct

Which of the following statements is/are true about Redis?

A. Data is always manipulated in the main computer memory (RAM).

B. It stores data on the disk by default.

C. Data cannot be reconstructed back.

D. Data is stored as key value pairs.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	Redis is an in-memory data store. Every read and every write is performed on the copy held in RAM. The word <i>always</i> is correct here, which is exactly what makes Redis so fast.
B	FALSE	This is the opposite of A, so it cannot be true at the same time. The disk is used only for optional persistence (a snapshot file, or an append-only log). Redis's home, by design, is memory – not the disk.
C	FALSE	Data <i>can</i> be reconstructed. If persistence is enabled, Redis reloads the snapshot or replays the append-only log after a restart and rebuilds everything in RAM (Figure 5.3).
D	TRUE	Redis is the textbook example of a key-value store: one text key points to one value (Figure 5.2).

ANSWER TO QUESTION 3

A and D

Confirmed by the green "Answer is Correct" banner in your screenshot.

TRAP BOX – HOW TO SPOT THE FAKE OPTION INSTANTLY

Options A and B contradict each other. In a multiple-select question, when you see a pair like that, only one of them can be on the answer list. Decide which side of the pair matches the definition of the tool, and the other one is automatically out.

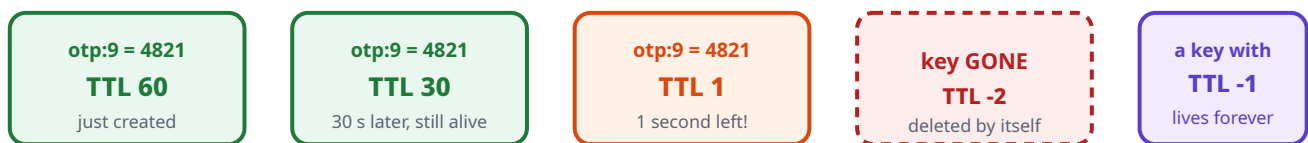
5.5 Two more Redis features you must know for Q4

Feature 1: expiry, also called TTL

STORY BOX • THE MILK PACKET

A milk packet has a "use before" stamp. Nobody has to throw it away by hand; the date decides. In Redis you may stamp any key with a countdown: "delete yourself automatically after 60 seconds". That countdown is called the **TTL**, short for **Time To Live**. It is perfect for one-time passwords, login sessions and cached pages.

A key with EXPIRE otp:9 60 – the countdown in action



THE THREE TTL NUMBERS THAT EXAMS LOVE TO SWAP

TTL = 1	a positive number = seconds remaining. TTL 1 means it dies in ONE second.
TTL = -1	the key exists but has NO expiry set. This is the "never expires" case.
TTL = -2	there is no such key any more (already expired or never created).

Figure 5.4 – TTL values. Confusing 1 with -1 is the single most common Redis mistake in exams.

Feature 2: authentication

STORY BOX • THE LOCK ON THE NOTEPAD

A notepad lying open on a desk can be read by anyone walking past. Redis can be given a **password** (the `requirepass` setting), and newer versions can even create separate users with different rights (this is called ACL). So yes, Redis has authentication features – they are simply switched off in a fresh install, which is why you must remember to turn them on.

Question 4

1 mark • multiple correct

Which of the following is/are true for Redis?

- A. Redis supports multiple data types.
- B. It is possible to set expiration time of data.
- C. Authentication features are available.
- D. TTL value of 1 implies that data will not expire.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	Strings, lists, hashes, sets, sorted sets, and more (bitmaps, streams, HyperLogLog). See Figure 5.2. Redis is often called a "data structure server" precisely for this reason.
B	TRUE	That is the TTL / EXPIRE feature from Figure 5.4.
C	TRUE	Password protection via requirepass , plus proper user accounts (ACLs) in Redis 6 and later.
D	FALSE	This is the trap. TTL 1 means "one second left to live" – the data expires almost immediately. The value that means "will not expire" is -1 .

ANSWER TO QUESTION 4

A, B and C

Confirmed by the green banner in your screenshot. D is false because "never expires" is TTL = -1, not TTL = 1.

Celery: giving slow jobs to a helper

6.1 The problem Celery solves

Go back to Q2 for a second. One slow request made another user wait 10 seconds for nothing. Now imagine the slow job is not a fake `sleep` but something real: "generate a 200-page PDF report and email it". That could take 3 minutes. Should the user's browser sit there spinning for 3 minutes? Obviously not.

STORY BOX • THE TAILOR'S RECEIPT

You give cloth to a tailor and ask for a shirt. The tailor does not make you stand at the counter for two days. He writes your job in his **order book**, hands you a **receipt**, and says "come on Friday". You leave immediately and get on with your life. Meanwhile his assistants pick jobs from the order book one by one and stitch them.

- The order book = the **broker** (a queue of pending jobs).
- The assistants = **Celery workers**, separate programs doing the slow work.
- Your receipt = the **task id**, which you can use later to ask "is it done?".
- The finished-shirt shelf = the **result backend**, where answers are kept.

DEFINITION BOX

Celery is a distributed task queue for Python. Your web app hands a slow job to a queue and replies to the user at once. Independent **worker** processes – possibly on completely different machines – pick jobs off the queue and run them. This style of work is called **asynchronous** or **background** processing.

6.2 The Celery picture you should be able to draw from memory

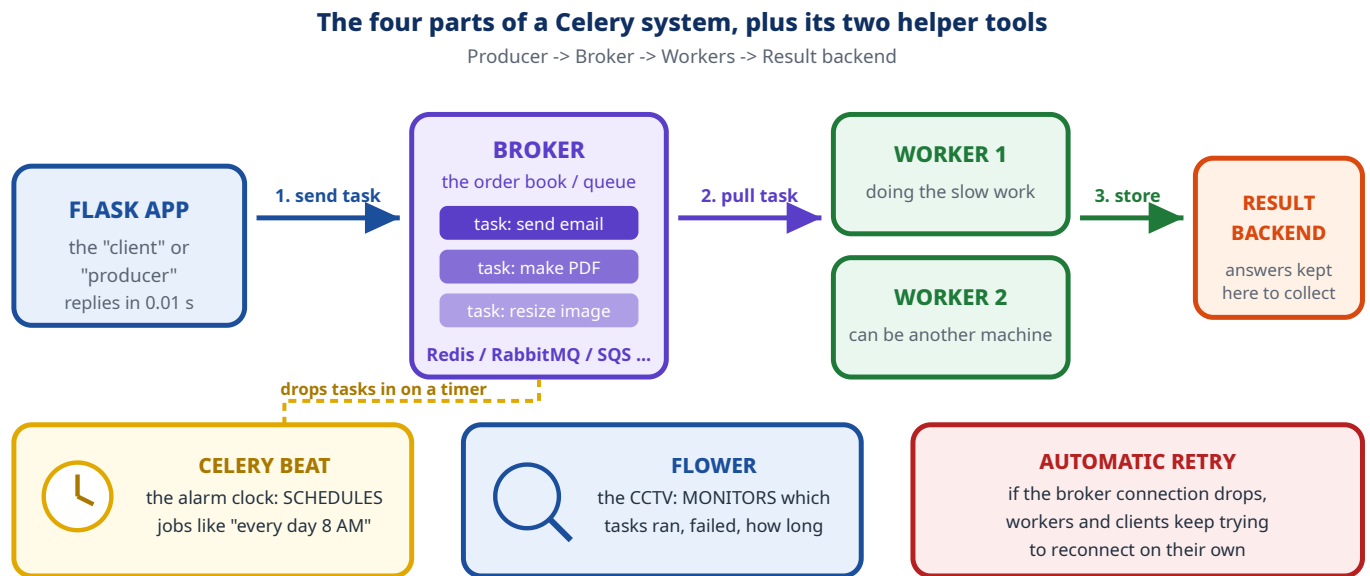


Figure 6.1 – Celery end to end. Learn the four boxes in the top row and the three cards below; every Celery exam question comes from this one picture.

6.3 Which brokers can Celery use?

This is the point option C attacks. Redis and RabbitMQ are the two **most common** choices, and RabbitMQ is the default recommendation – but they are not the only ones. Celery also supports **Amazon SQS**, and has further transports such as Zookeeper and consul, plus a filesystem transport for local testing.

REMEMBER THE SHAPE OF THE SENTENCE, NOT JUST THE FACTS

Words like **"only"**, **"always"** and **"never"** are almost always the sign of a false option. Real engineering tools are built to be pluggable. Whenever you read "X and Y are the only supported ...", your first instinct should be suspicion.

Question 5

1 mark • multiple correct

Which of the below is true for Celery?

- A. Celery systems can consist of multiple brokers.
- B. Workers and clients will automatically retry if connection is lost.
- C. Redis and RabbitMQ are the only supported brokers with Celery.
- D. It has features to monitor and schedule tasks.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	A Celery deployment is not limited to one broker. You can point workers at more than one broker, which is how large systems stay available if one broker dies and how they split traffic between queues.
B	TRUE	Celery is explicitly built to survive network trouble: clients and workers retry the broker connection automatically instead of crashing. This property is called fault tolerance .
C	FALSE	The word only kills it. Amazon SQS is also supported, along with several experimental transports. Redis and RabbitMQ are merely the popular pair.
D	TRUE	Two well-known features: Flower (a web dashboard to monitor tasks and workers) and Celery Beat (a scheduler for periodic tasks, like a cron job). See the bottom row of Figure 6.1.

ANSWER TO QUESTION 5

A, B and D

Confirmed by the green banner in your screenshot. Only C is false, because of the word "only".

Concurrency vs Parallelism

These two words sound like synonyms. They are not, and Q6 and Q7 exist purely to see whether you know the difference. Ten minutes here will earn you two marks forever.

7.1 The two stories

STORY BOX • CONCURRENCY = ONE COOK, THREE DISHES

Amma is cooking rice, dal and a sabzi with only two hands. She puts the rice on to boil, and while it boils she chops vegetables. Then she stirs the dal, then goes back to the vegetables. She is **never doing two things in the same instant**, but all three dishes are **in progress at the same period of time**, and all three land on the table together.

That is **concurrency**: *dealing with* many tasks over an overlapping stretch of time, by switching between them.

STORY BOX • PARALLELISM = THREE COOKS, THREE DISHES

Now Amma calls two neighbours. Three people stand at three stoves. At any single instant, three dishes are genuinely being cooked at once.

That is **parallelism**: *actually doing* many tasks in the very same instant. It needs extra hands – which for a computer means extra **CPU cores**.

DEFINITION BOX – THE EXAM WORDING

- **Concurrency**: multiple tasks make progress over **overlapping time periods**, not necessarily at the same instant.
- **Parallelism**: multiple tasks execute **simultaneously**, literally at the same instant, which requires multiple cores or processors.
- **Context switching**: one core rapidly pausing task A, saving its place, and resuming task B. This is *how* a single core fakes doing many things at once.

7.2 The picture that settles both questions

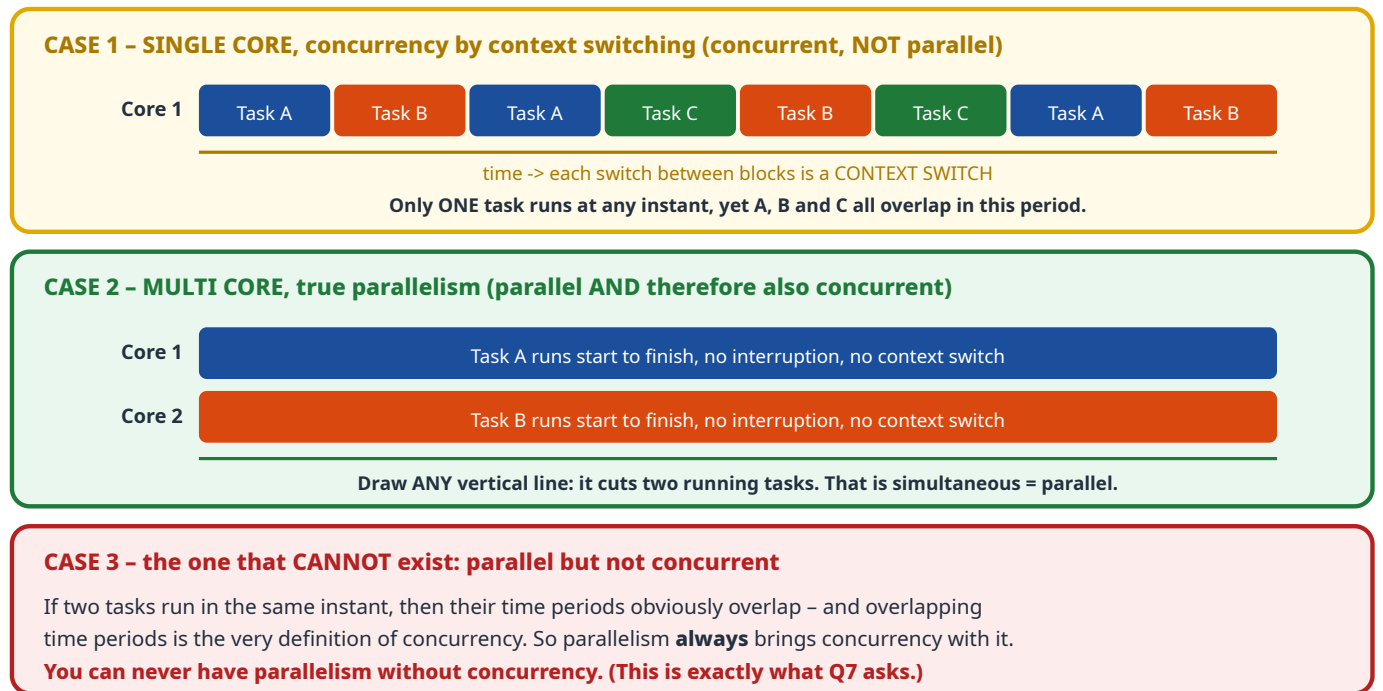


Figure 7.1 – Three cases. Case 1 and Case 2 are both real; Case 3 is impossible.

7.3 The relationship, as a Venn diagram

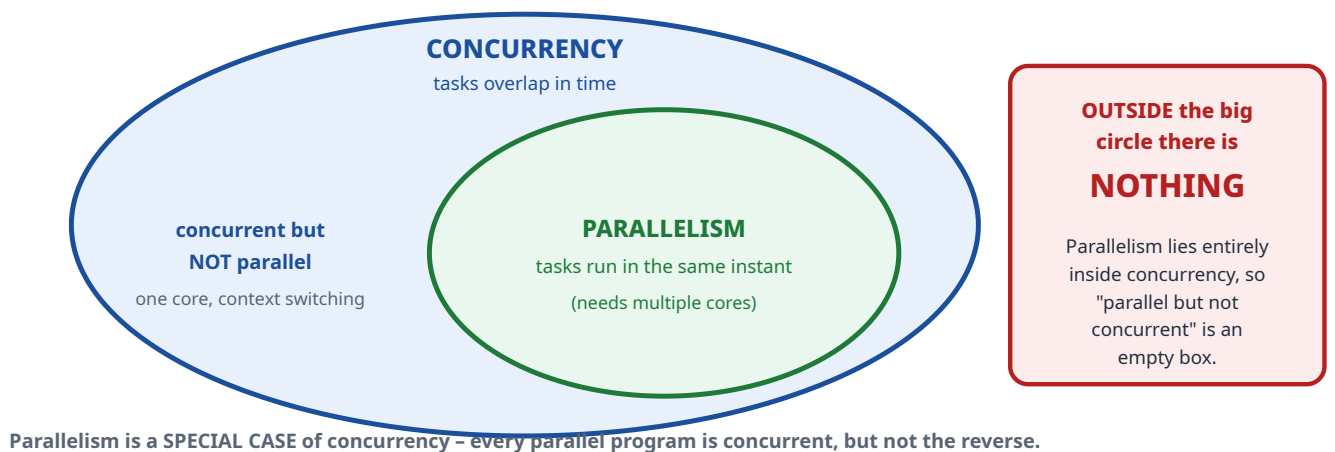


Figure 7.2 – The small circle sits inside the big one. Memorise this shape and both questions become easy.

7.4 The four combinations, as a table

Can this exist?	Possible?	Example
Concurrent and parallel	YES	Two cores each running one task, both busy the whole time.
Concurrent but not parallel	YES	One core context-switching between three tasks. Amma cooking alone.
Parallel but not concurrent	NO	Impossible. Running in the same instant means the time periods overlap, which is already concurrency.
Neither	YES	One task, start to finish, then the next one. Plain sequential code – exactly what <code>threaded=False</code> gave us in Q2.

Question 6

1 mark • pick the FALSE one

Which of the following statements is **false** regarding concurrency?

- A. The concurrency is achieved by executing multiple tasks overlapping in time periods, but not always simultaneously.
- B. The concurrency can be achieved in both "single-core" and "multi-core" environments.
- C. The concurrency is **always** achieved via "context switching" in a multi-core environment.
- D. Concurrency and Parallelism are not mutually exclusive.

READ THE INSTRUCTION TWICE

This question asks for the **false** statement. So the option you tick is the *wrong* statement. Students who solve it correctly but tick a true statement out of habit still lose the mark.

Option-by-option reasoning

Option	True or false	Why
A	TRUE	This is the standard definition, and the phrase "not always simultaneously" is what makes it correct. Case 1 of Figure 7.1.
B	TRUE	On a single core you get concurrency through context switching; on multiple cores you get it through parallel execution. Both roads lead to concurrency.
C	FALSE the answer	The word "always" is the error. In a multi-core environment two tasks can run on two different cores at the same time, so no context switching is needed at all (Case 2 of Figure 7.1). Context switching is <i>one</i> way to get concurrency – the way a single core must use – not the only way.
D	TRUE	"Mutually exclusive" would mean they can never happen together. But they can: the overlap region of Figure 7.2 is real and common. So saying they are not mutually exclusive is correct.

ANSWER TO QUESTION 6

Option C is the false statement.

Confirmed by the green banner in your screenshot.

Question 7

1 mark • pick the FALSE one

Which of the following statements is **false** regarding concurrency and parallelism?

- A. The parallelism can be achieved in a "multi-core" environment.
- B. It is possible to achieve parallelism **without** concurrency.
- C. The parallelism is achieved by executing multiple tasks simultaneously.
- D. It is possible to achieve concurrency without parallelism.

Option-by-option reasoning

Option	True or false	Why
A	TRUE	Multiple cores are exactly what parallelism requires. Three cooks, three stoves.
B	FALSE the answer	This is Case 3 in Figure 7.1 and the empty box in Figure 7.2. If two tasks execute simultaneously, their time periods overlap by definition, and overlapping time periods <i>is</i> concurrency. So parallelism cannot exist on its own. Parallelism always implies concurrency.
C	TRUE	The word "simultaneously" is precisely the definition of parallelism.
D	TRUE	The reverse direction <i>is</i> allowed: a single core switching between tasks is concurrent with zero parallelism. Amma cooking alone.

ANSWER TO QUESTION 7

Option B is the false statement.

Confirmed by the green banner in your screenshot.

ONE LINE TO CARRY INTO THE EXAM

Parallel implies concurrent. Concurrent does not imply parallel. Read that arrow in one direction only, and Q6 and Q7 can never trick you again.

Message queues and message brokers

8.1 What is a message queue?

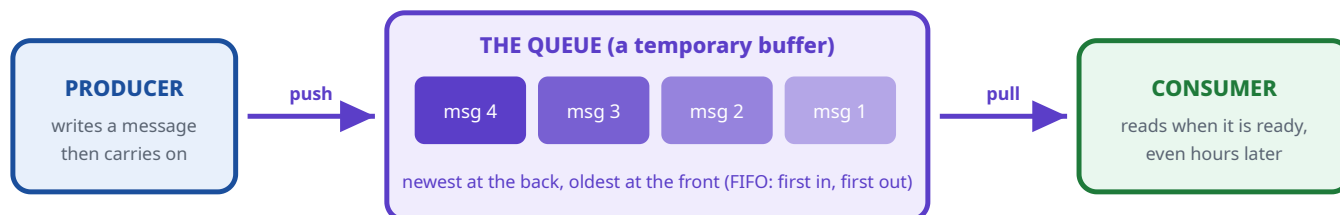
STORY BOX • THE LETTER BOX

You want to send your cousin a letter. You do **not** need to phone her first and check whether she is at home. You drop the letter in the post box and walk away. The letter waits safely in the box until the postman picks it up. Your cousin reads it whenever she gets home.

A **message queue** is that post box for programs.

- The person posting = the **producer** or **sender**.
- The letter = the **message** (a small packet of data).
- The post box = the **queue**, which acts as a **buffer** holding messages temporarily.
- Your cousin = the **consumer** or **receiver**.

A message queue is a BUFFER that decouples sender from receiver



Why this is powerful: the producer and the consumer never have to be awake at the same moment.

If the consumer crashes, messages simply wait in the buffer. If a burst of 10 000 messages arrives, the queue absorbs the shock instead of the consumer collapsing. This separation is called DECOUPLING.

Figure 8.1 – Producer, queue, consumer. The queue is a buffer that stores messages temporarily.

8.2 Point-to-point vs publish-subscribe

Q8 mentions "point-to-point messaging". There are two delivery styles, and you must be able to tell them apart.

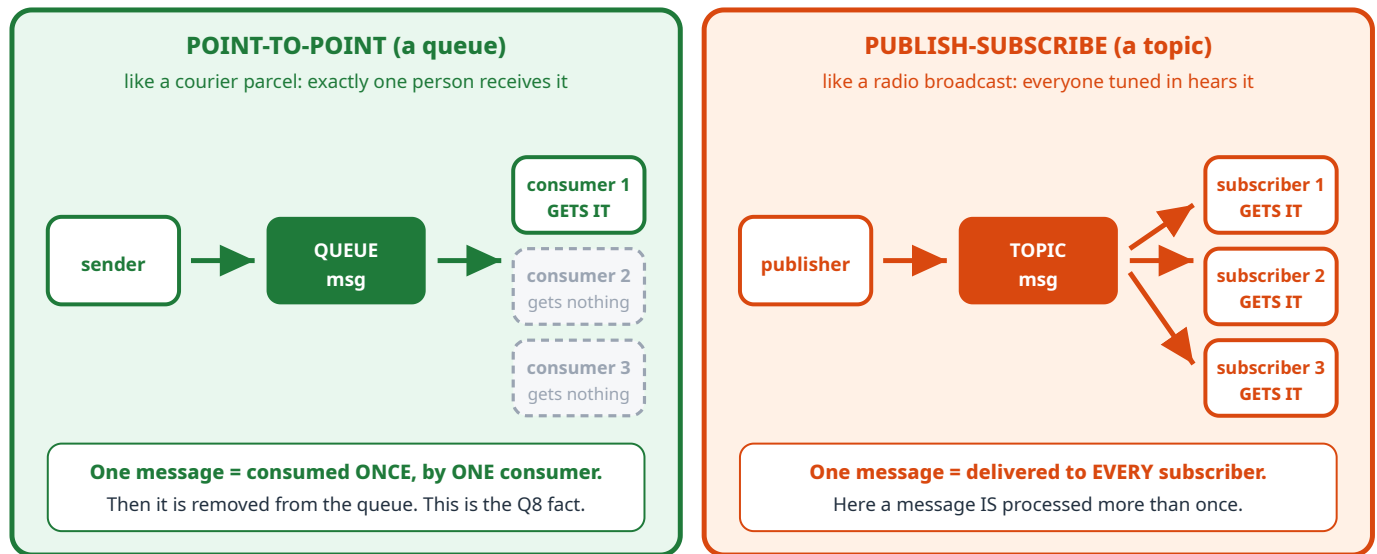


Figure 8.2 – The two messaging styles. Q8 is about the left-hand one.

Question 8

1 mark • multiple correct

Which of the following statement(s) is/are correct regarding message queue?

- A. Message queue provides communication between components in a distributed architecture.
- B. It provides a buffer which temporarily store messages.
- C. In general, a message in message queue is processed **more than once** in point-to-point messaging.
- D. In general, a message in message queue is processed **only once** in point-to-point messaging.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	"Distributed architecture" just means the software is split into pieces that may sit on different machines. Letting those pieces talk to each other without being directly wired together is the main job of a message queue.
B	TRUE	A queue holds messages temporarily until a consumer takes them – that is the textbook definition of a buffer (Figure 8.1). Note the word <i>temporarily</i> : a queue is not long-term storage like a database.
C	FALSE	In point-to-point, once a consumer takes a message it is removed, so it is not processed again. "More than once" describes publish-subscribe, not point-to-point.
D	TRUE	Exactly the left panel of Figure 8.2: one message, one consumer, processed once. Note that C and D are direct opposites, so precisely one of them can be right.

ANSWER TO QUESTION 8

A, B and D

Confirmed by the green banner in your screenshot.

8.3 What is a message broker, and why does it save connections?

STORY BOX • THE POST OFFICE

A message queue is one post box. A **message broker** is the whole **post office**: it owns many boxes, sorts letters, decides which van carries what, retries failed deliveries, and keeps records.

Now think about what life would be like without a post office. To write to 4 friends you would need your own private road to each of their houses. And if all 4 of you wanted to write to each other, the town would need a spider web of private roads. With a post office, everyone builds just **one** road – to the post office.

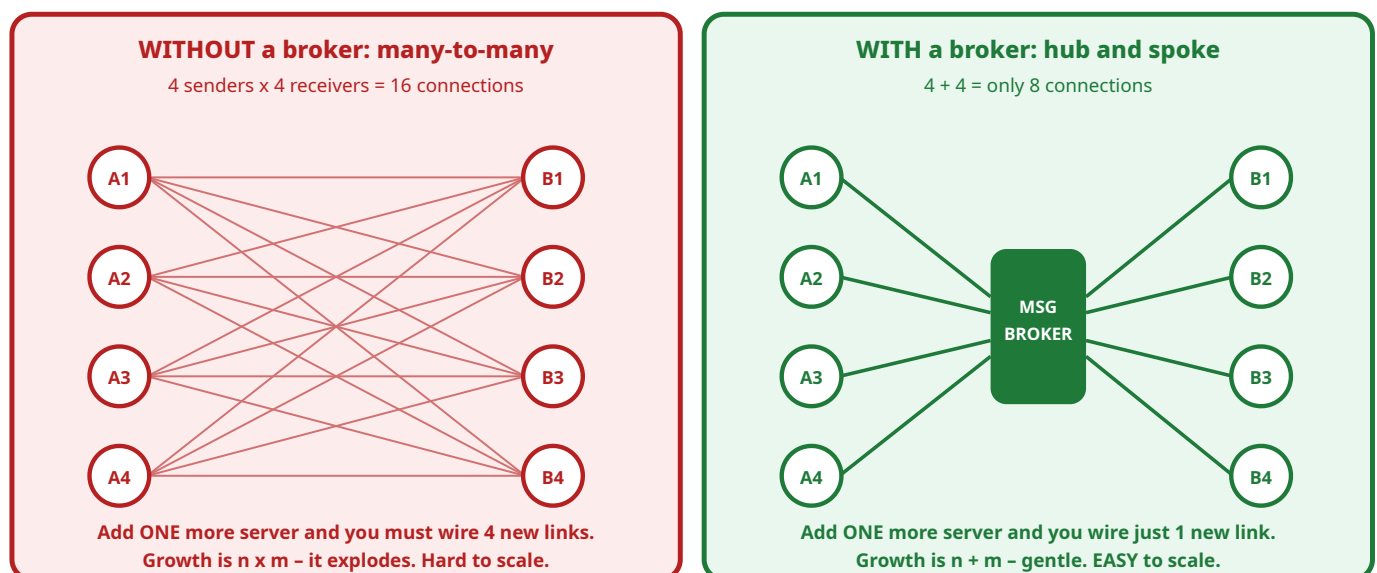


Figure 8.3 – The connection-count argument. This single picture answers both parts of Q9.

DO THE ARITHMETIC ONCE AND YOU WILL NEVER FORGET IT

With 10 senders and 10 receivers: direct wiring needs $10 \times 10 = 100$ connections. Through a broker: $10 + 10 = 20$. With 100 and 100 it is **10 000** versus **200**. Fewer connections means less code, fewer failures, and far easier scaling.

Question 9

1 mark • multiple correct

Which of the following is/are true regarding message broker?

- A. A message broker helps in reducing the number of connections between servers, if compared with many-to-many connections between servers.
- B. It does **not** allow easier scalability, if compared with many-to-many connections between servers.
- C. It can be used for batch processing of many similar types of requests.
- D. All of the above

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	16 connections become 8; 100 become 20. Exactly Figure 8.3.
B	FALSE	Read the little word not carefully. A broker <i>does</i> make scaling easier, because adding one more server costs one new connection instead of many. The statement claims the opposite, so it is false.
C	TRUE	Because messages pile up in a queue, a consumer can collect a whole batch and process them together – for example, sending 500 collected emails in one go. Batch processing is a standard broker use case.
D	FALSE	"All of the above" would require B to be true as well. Since B is false, D collapses automatically.

ANSWER TO QUESTION 9

A and C

Confirmed by the green banner in your screenshot.

TRAP BOX – THE TINY WORD "NOT"

Option B is the same fact as option A, just phrased negatively. Examiners love this. When you meet a negative option, rewrite it in your head as a positive sentence ("a broker *does* allow easier scalability"), decide whether *that* is true, and then flip your verdict once.

8.4 Synchronous vs asynchronous, the last idea you need

STORY BOX • PHONE CALL VS TEXT MESSAGE

A **phone call** is **synchronous**: both people must be free at the same moment, and you cannot do anything else while talking. If she does not pick up, your message does not get through at all.

A **text message** is **asynchronous**: you send it and immediately go back to your game. You do not need to know whether her phone is on, off, charging or out of signal. She replies whenever she can.

Message brokers are the text-message model. That is the whole of Q10.

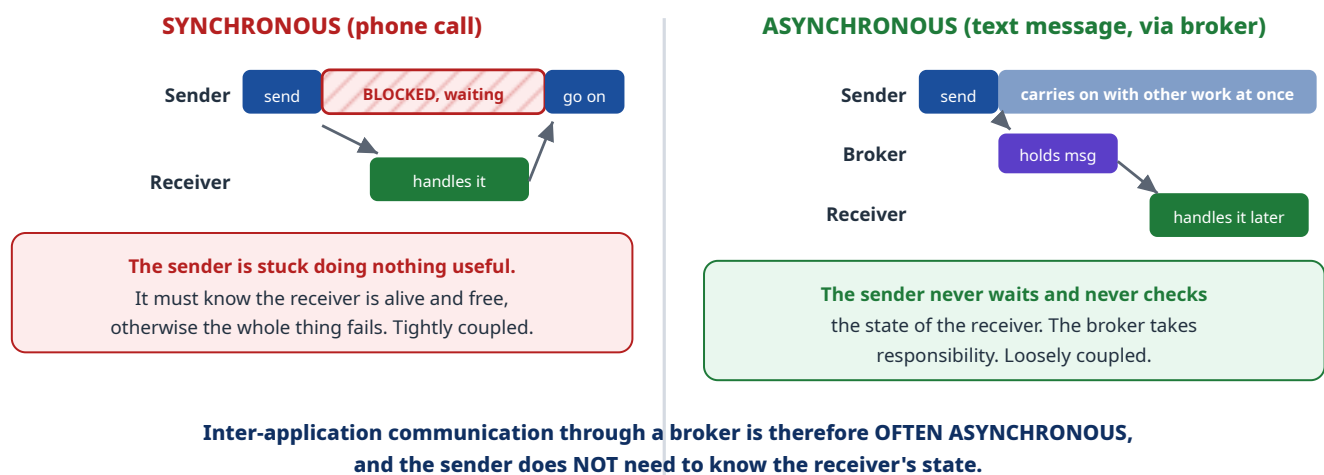


Figure 8.4 – Synchronous blocks the sender; asynchronous frees it. Brokers give you the second one.

Question 10

1 mark • multiple correct

Which of the following is/are true regarding message broker?

- A.** They serve as intermediary between applications, which allow senders to issue a message **without** knowing the state of the receiver.
- B.** They serve as intermediary between applications, which allow senders to issue a message **only after** knowing the state of the receiver.
- C.** Inter application communication is often **asynchronous**.
- D.** Inter application communication is often **synchronous**.

Option-by-option reasoning

Option	Verdict	Why
A	TRUE	This is the definition of a broker: an intermediary . You post the letter and walk away; the post office worries about the recipient. The sender needs no knowledge of the receiver's state.
B	FALSE	The exact opposite of A. If the sender had to check the receiver first, the broker would be pointless – you would be back to a phone call.
C	TRUE	Communication through a broker is asynchronous: send now, get handled later (Figure 8.4, right side). Note the honest word "often" rather than "always".
D	FALSE	The opposite of C. Synchronous messaging is the model brokers were invented to escape.

ANSWER TO QUESTION 10

A and C

Confirmed by the green banner in your screenshot.

PATTERN SPOTTING: THIS PAPER IS FULL OF OPPOSITE PAIRS

Q3 had A vs B. Q8 had C vs D. Q10 had A vs B *and* C vs D. In every case exactly one member of the pair is correct. Spot the pairs first, settle each pair with one clear fact, and a 4-option question shrinks to two easy decisions.

Answer key, cheat sheet and practice

9.1 Master answer key

Q	Topic	Answer	The one line that decides it
1	Flask, default threading	B 20, 40	<code>app.run()</code> is threaded by default, so both requests start together.
2	Flask, <code>threaded=False</code>	A 20, 50	One thread, so <code>/profile</code> waits 10 s first: $10 + 30 = 40$ s after 10:10:10.
3	Redis basics	A, D	Always works in RAM; stores key-value pairs. Disk is only a backup.
4	Redis features	A, B, C	Many data types, TTL expiry, authentication. TTL 1 means 1 second, not "never".
5	Celery	A, B, D	Multiple brokers, auto retry, Flower + Beat. "Only Redis and RabbitMQ" is false.
6	Concurrency (false one)	C	Multi-core concurrency does not <i>always</i> need context switching.
7	Concurrency vs parallelism (false one)	B	Parallelism without concurrency is impossible.
8	Message queue	A, B, D	It is a temporary buffer; in point-to-point a message is consumed once.
9	Message broker, connections	A, C	$n + m$ instead of $n \times m$, and batch processing is possible. B has a stray "not".
10	Message broker, coupling	A, C	Sender need not know the receiver's state; communication is asynchronous.

REMINDER ABOUT Q2

Q2 is the only place where this key differs from the option that was selected in your screenshot. Q1 and Q2 are deliberately built as a pair: the examiner changes `threaded=True` (the default) to `threaded=False` so that the answer must move from **20, 40** to **20, 50**. If you are ever unsure in the real exam, redraw Figure 4.3 on your rough sheet; the picture cannot lie.

9.2 The one-page cheat sheet

Flask timing questions

- `app.run()` and `app.run(debug=True)` = **threaded**. Requests overlap.
- `app.run(threaded=False)` = **one at a time**. Requests queue up.
- Formula: answer second = (start second + waiting time + own sleep) mod 60.
- The clock is read **after** the sleep, so waiting time counts.
- Console order = order of **arrival** of replies, not the order the code was typed.
- `jsonify` , status `200` , the CORS header and `debug=True` never change timing.

Redis

- In-memory key-value store. All work happens in **RAM**; disk is only for backup.
- Data types: string, list, hash, set, sorted set (and more).
- Persistence: RDB snapshot or AOF log, so data **can** be reconstructed.
- TTL: positive = seconds left, **-1 = no expiry**, **-2 = key does not exist**.
- Has authentication (`requirepass` , and ACL users in Redis 6+).
- Typical uses: caching, sessions, rate limiting, leaderboards, Celery broker.

Celery

- Four pieces: **producer** (your app) • **broker** (queue) • **worker** • **result backend**.
- Brokers: RabbitMQ, Redis, Amazon SQS and others – **not** just the first two.
- **Flower** = monitoring dashboard. **Celery Beat** = periodic scheduler.
- Clients and workers retry automatically when the connection drops.
- Can use more than one broker in one system.

Concurrency and parallelism

- **Concurrency**: overlapping time periods, not necessarily the same instant.
- **Parallelism**: the same instant, needs multiple cores.
- **Parallel implies concurrent. Concurrent does not imply parallel.**
- Concurrency works on single-core (context switching) and multi-core (real parallelism).
- They are **not** mutually exclusive.

Message queue and message broker

- Queue = temporary **buffer**; it decouples producer from consumer.
- **Point-to-point**: one message, one consumer, processed **once**.
- **Publish-subscribe**: one message, every subscriber, processed many times.
- Broker turns **n x m** connections into **n + m**, so scaling gets easier.
- Broker communication is **asynchronous**; the sender does not check the receiver's state.
- Brokers enable **batch processing** of many similar requests.

9.3 Exam technique: the keyword radar

Six of the ten questions in this paper are decided by a single word. Train your eye on these:

Word you spot	What it usually means	Where it happened in this paper
always	likely FALSE	Q6 option C: "always achieved via context switching".
only	likely FALSE	Q5 option C: "Redis and RabbitMQ are the only supported brokers".
not (buried mid-sentence)	flip your verdict	Q9 option B: "does not allow easier scalability".
often, in general, not always	likely TRUE	Q10 option C, Q8 options C and D, Q6 option A. Careful wording is usually honest wording.
All of the above	check every option first	Q9 option D: dies because option B is false.
false in the question stem	tick the wrong statement	Q6 and Q7. Underline the word "false" before you start.

A 30-SECOND ROUTINE FOR ANY MULTIPLE-SELECT QUESTION

1. Underline whether the stem wants TRUE or FALSE statements. **2.** Hunt for opposite pairs and settle each pair. **3.** Circle every *always* / *only* / *never* / *not*. **4.** Then, and only then, choose.

9.4 Practice questions

Cover the answers with your hand. If you can do these seven, you have genuinely learned Week 9 and not just memorised a key.

- P1.** Same `app.py` and `index.html` as Q1 (threaded), but the user opens the page at **10:10:45**. What is logged?
- P2.** Same as Q2 (`threaded=False`), page opened at **10:10:45**. What is logged?
- P3.** In the Q2 code, the two `fetch` lines are swapped so `/profile` is requested first. Page opened at 10:10:10. What is logged, and in what order?
- P4.** `TTL mykey` returns `-1` . Will the key be deleted in one second?
- P5.** A single-core machine runs three tasks that all finish within the same minute. Is that concurrency? Is it parallelism?
- P6.** You have 6 sender services and 5 receiver services. How many connections with direct wiring, and how many through a broker?
- P7.** A message is placed in a point-to-point queue with 3 consumers attached. How many consumers process it?

ANSWERS TO THE PRACTICE QUESTIONS

- P1. 55, 15.** Threaded, so both start at 10:10:45. `/` finishes at 10:10:55, second = **55**. `/profile` finishes 30 s later at 10:11:15, and the seconds hand has wrapped past 59, so second = **15**. ($75 \bmod 60 = 15$. This is the wrap-around case.)
- P2. 55, 25.** One thread: `/` finishes at 10:10:55 (second 55). `/profile` starts only then and runs 30 s, finishing at 10:11:25, so second = **25**. ($45 + 10 + 30 = 85$; $85 \bmod 60 = 25$.)
- P3. 40, then 50.** `/profile` is served first: 0 to 30 s, finishing at 10:10:40, so it logs **40**. Then `/` runs from $t = 30$ to $t = 40$, finishing at 10:10:50, so it logs **50**. The 40 is printed first because it arrived first, even though it belongs to the second `fetch` in the code.
- P4. No.** `TTL -1` means the key exists with *no* expiry set, so it stays until something deletes it. `TTL 1` would be the "gone in one second" case.
- P5. Concurrency yes, parallelism no.** On one core the tasks take turns through context switching, so their time periods overlap but no two ever run in the same instant.
- P6.** Direct wiring: $6 \times 5 = \mathbf{30}$ connections. Through a broker: $6 + 5 = \mathbf{11}$.
- P7. Exactly one.** In point-to-point the first consumer to take the message removes it from the queue; the other two never see it. (In publish-subscribe, all three would receive it.)

9.5 Glossary of every term used in this paper

Term	Meaning in one sentence
Client	The program that asks; usually your browser.
Server	The program that answers; here, Flask on port 5000.
Route	The path part of a URL, such as <code>/</code> or <code>/profile</code> .

localhost / 127.0.0.1	A nickname meaning "this very computer".
Port	A numbered door on a computer; Flask's development server uses 5000.
CORS header	A permission slip letting a page read data from another address.
Promise	A token in JavaScript standing for a result that will arrive later.
Thread	One worker inside a program, able to run one job at a time.
Context switching	One core pausing task A and resuming task B, very rapidly.
Concurrency	Several tasks progressing over overlapping periods of time.
Parallelism	Several tasks running in the very same instant, on several cores.
Blocking	Being forced to wait, unable to do anything else meanwhile.
Synchronous	Send and wait for the reply before continuing (a phone call).
Asynchronous	Send and carry on immediately (a text message).
In-memory store	A database that keeps working data in RAM, such as Redis.
Key-value pair	One name (the key) pointing to one piece of data (the value).
TTL	Time To Live: seconds until a key deletes itself.
Persistence	Copying in-memory data to disk so it survives a restart.
Task queue	A list of pending background jobs, e.g. managed by Celery.
Worker	A separate process that pulls jobs from a queue and runs them.
Broker	The middleman service holding and routing messages between apps.
Result backend	Storage where a finished task's answer is kept for collection.
Producer / Consumer	The one who puts messages in / the one who takes them out.
Buffer	Temporary holding space that absorbs bursts of traffic.
Decoupling	Arranging things so two parts need not know about each other.
Point-to-point	One message consumed by exactly one consumer.
Publish-subscribe	One message delivered to every subscriber.
Fault tolerance	Continuing to work correctly when parts of the system fail.
Batch processing	Collecting many similar jobs and doing them in one sweep.

9.6 Final word

IF YOU REMEMBER ONLY FIVE SENTENCES FROM THIS WHOLE BOOK

1. Flask's `app.run()` is **threaded by default**; `threaded=False` makes it queue.
2. The clock is read **after** the sleep, so waiting time is part of the answer.
3. Redis works in RAM, stores key-value pairs, and **TTL -1** means "never expires".
4. **Parallel implies concurrent**, never the other way round.
5. A broker turns **$n \times m$** connections into **$n + m$** , and lets senders fire and forget.

*Prepared as a step-by-step teaching solution for Week 9 – Graded Assignment 9.
Work through the figures with a pencil in hand; that is where the learning actually happens.*

NOTES ON SOURCES

The technical facts in this book come from the official documentation of the tools discussed – Flask and Werkzeug (flask.palletsprojects.com), Redis (redis.io/docs) and Celery (docs.celeryq.dev) – together with the Week 9 lecture material of the course. The specific detail that Flask's `app.run()` sets `threaded=True` by default from Flask 1.0 onwards was cross-checked against community documentation on [Stack Overflow](https://stackoverflow.com). All explanations, analogies and diagrams here were written and drawn for this document. Content was rephrased for compliance with licensing restrictions.